High Level Design on Multi Insurance Creator

Course Name: GEN AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Yash Shrivastava	EN22CS3011111
2.	Vishwas Bhargav	EN22CS3011095
3.	Yashi Harne	EN22CS3011116
4.	Dixika Verma	EN22ME304027
5.	Anvesha Wankhade	EN22ME304014

Group Name: GROUP 07D10

Project Number: GAI-20

Industry Mentor Name:

University Mentor Name: Prof. Divya Kumawat

Academic year : 2025-26

MEDICAPS UNIVERSITY

Datagami
"Lead Digital Technology"

Table of Contents

1. **Introduction.**
    - 1.1. Scope of the document.
    - 1.2. Intended Audience
    - 1.3. System overview.
2. **System Design.**
    - 2.1. Application Design
    - 2.2. Process Flow.
    - 2.3. Information Flow.
    - 2.4. Components Design
    - 2.5. Key Design Considerations
    - 2.6. API Catalogue.

3, Data Design.

- 3.1. Data Model
- 3.2. Data Access Mechanism
- 3.3. Data Retention Policies
- 3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements

- 7.1. Security Aspects
- 7.2. Performance Aspects

8. **References**

1. Introduction

The Multimodal Insurance Creator leverages generative AI to transform textual insurance concepts into both detailed narratives and high-quality visual representations. This document provides a structured overview of the system, its purpose, and intended usage.

1.1 Scope of the Document

This document describes the high-level design for deploying a MERN stack Insurance application on a Kubernetes cluster. It covers the architecture, component interactions, data design, interfaces, and non-functional requirements such as security and performance. The goal is to build a scalable, highly available, and maintainable system that supports zero-downtime updates using Kubernetes rolling deployments. This design applies to both local environments (Minikube) and cloud environments (e.g., AWS EKS).

1.2 Intended Audience

This document outlines the objectives, design, and functionality of the Multimodal Insurance Creator. It covers:

- Integration of Large Language Models (LLMs) and image generation APIs to create multimodal content.
- System architecture, workflows, and data handling for generating and managing outputs.
- Guidelines for using the system to produce coherent text and visuals for professional and educational applications.

Exclusions: Detailed internal implementations of third-party AI models are outside the scope; focus is on system-level orchestration and application use.

1.3 System Overview

The Multimodal Insurance Creator converts textual prompts into multi-dimensional outputs that combine narrative and visual information:

- Input Layer: Accepts textual prompts describing insurance concepts or scenarios.
- Processing Layer:
  - ➢ Text-to-Text: Generates structured, context-aware narratives.
  - ➢ Text-to-Image: Produces visual representations such as risk infographics or conceptual diagrams.
- Output Layer: Delivers integrated multimodal content suitable for presentations, reports, and educational materials.
- Storage & Retrieval: Optional vector database enables indexing and efficient retrieval of generated outputs.

This system demonstrates the synergy of text and image generation models to provide comprehensive, professional-grade insights into insurance topics.

2. System Design

2.1 Application Design

This diagram represents the architecture of a multimodal insurance application that generates both textual and visual outputs.

- **User**

The user provides input such as an insurance query, policy request, or risk description.

- **API Layer**

The API acts as an interface between the user and the system. It handles incoming requests and sends them to the AI pipeline, then returns the generated response back to the user.

- **AI Pipeline**

This is the core component that processes the user input. It analyzes the request and routes it to appropriate AI services for generating results.

- **LLM API (Large Language Model)**

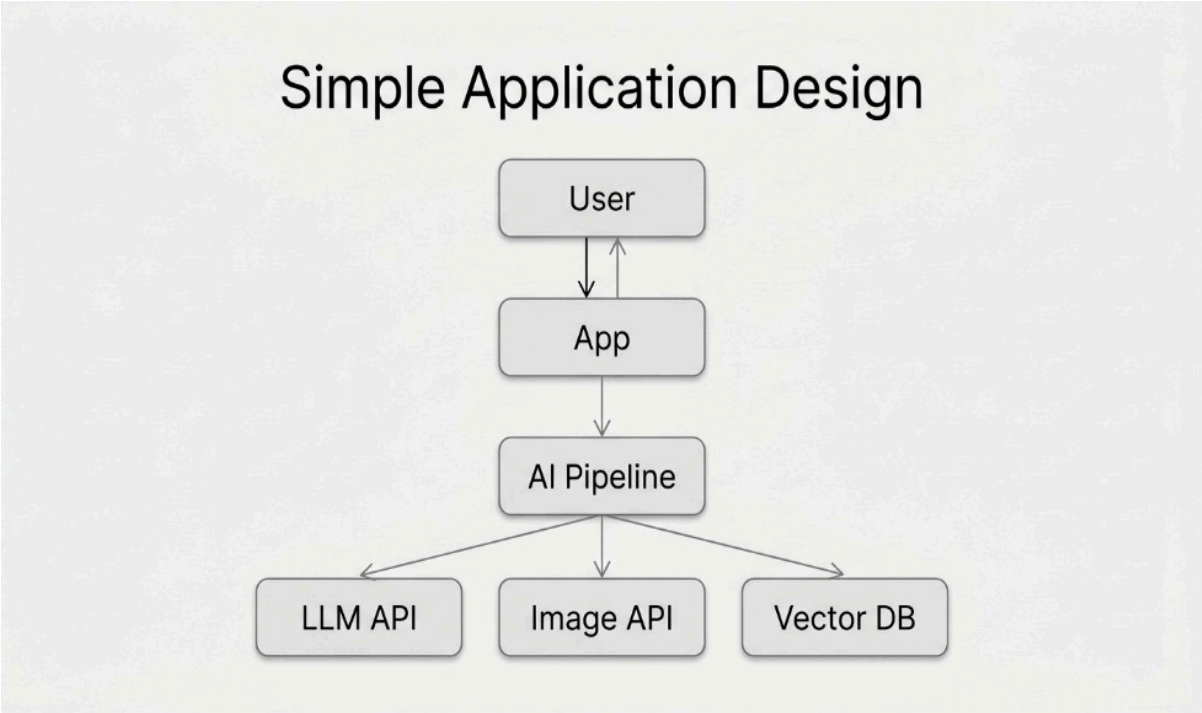
Used to generate insurance-related textual content such as policy explanations, claim details, and risk analysis.

- **Image API**

Generates visual representations like risk diagrams, insurance infographics, or claim illustrations based on user input.

- **Vector Database**

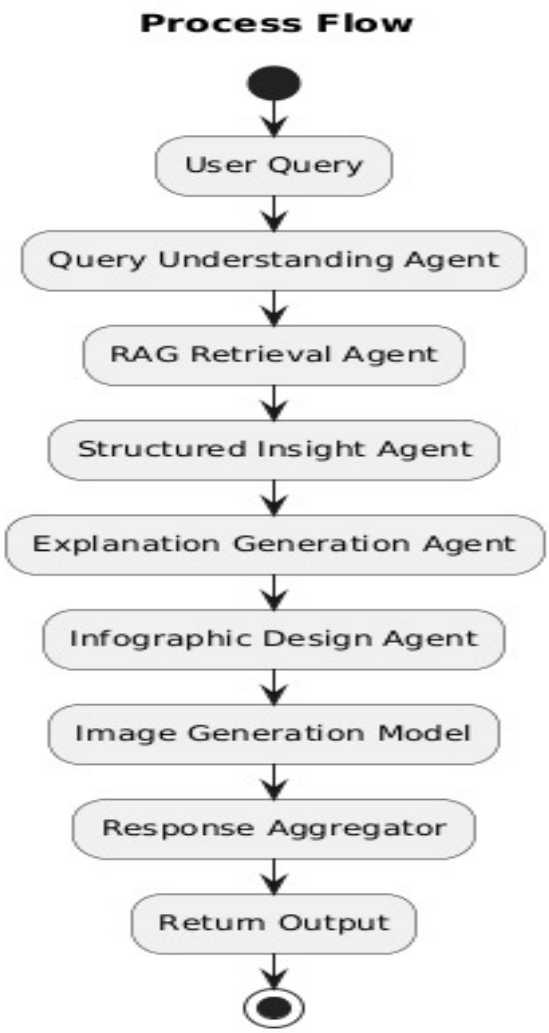
Stores insurance-related knowledge in the form of embeddings. It helps retrieve relevant information to improve response accuracy and context understanding.



2.2 Process Flow

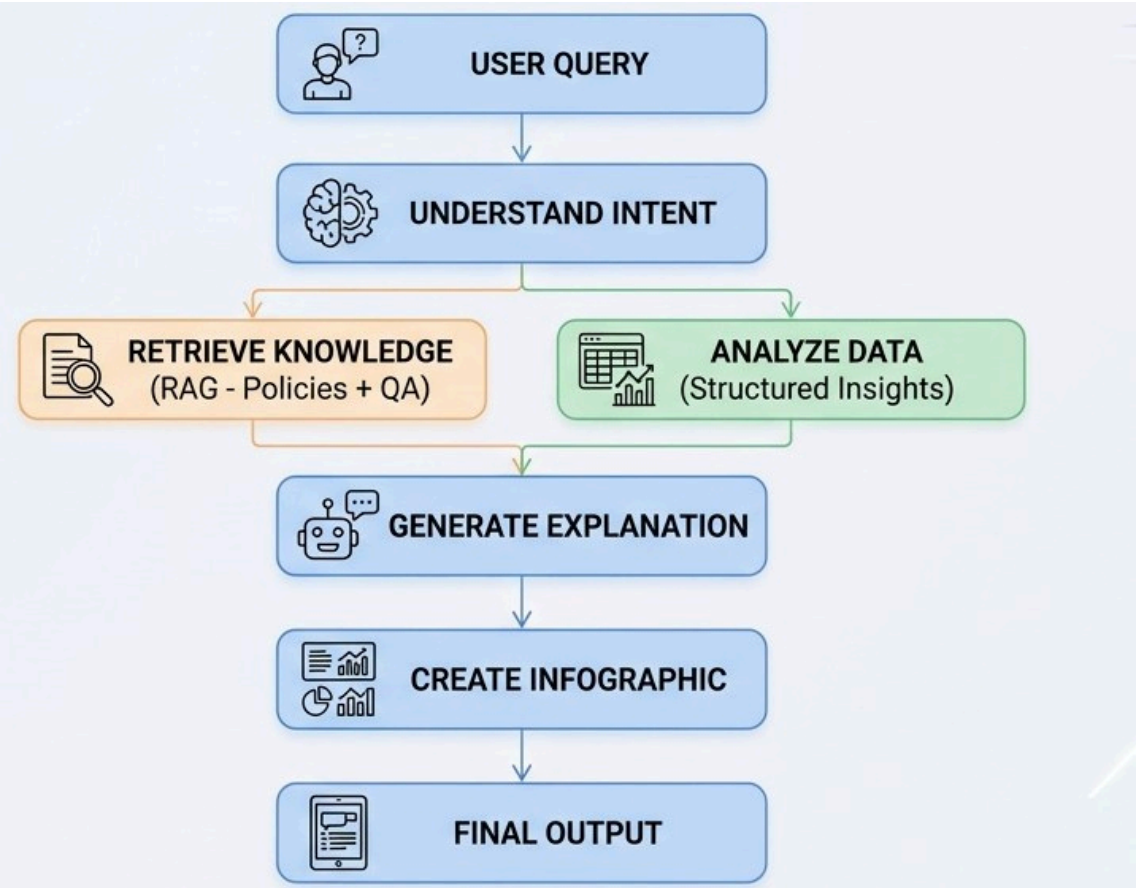
- Starts with a start node (black circle)
- User Query (user inputs request)
- Query Understanding Agent (interprets the query)

- RAG Retrieval Agent (fetches relevant information)
- Structured Insight Agent (analyzes structured data)
- Explanation Generation Agent (creates explanation)
- Infographic Design Agent (designs visual output)
- Image Generation Model (generates visuals/images)
- Response Aggregator (combines all outputs)
- Return Output (final response to user)
- Ends with an end node (circle)



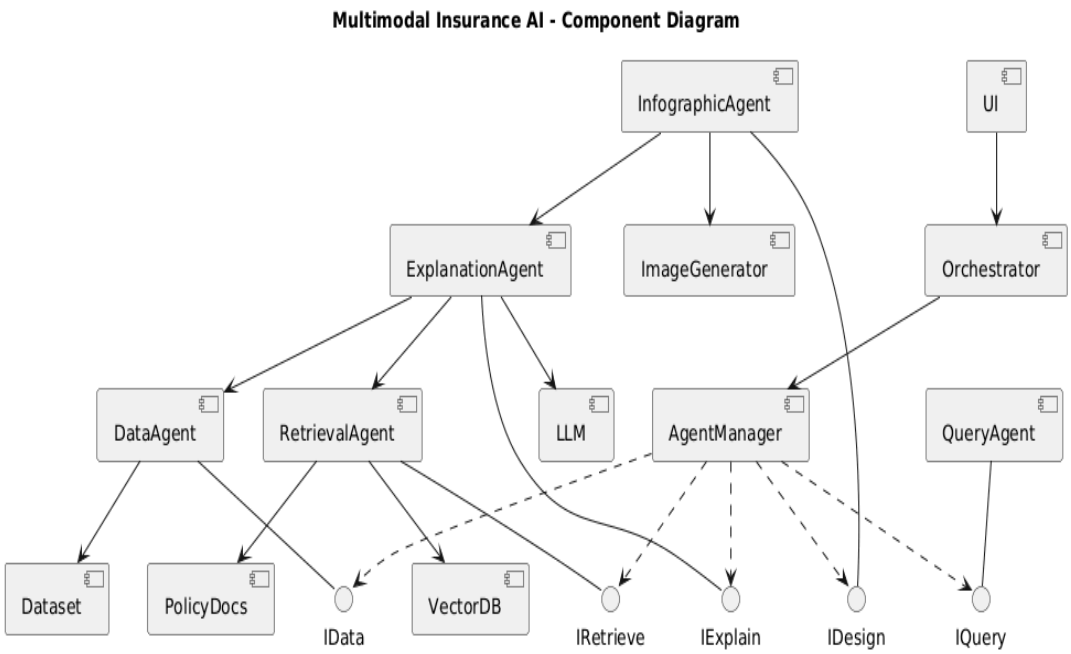
2.3 Information Flow

- **Title**: Multimodal Insurance AI System Information Flow
- **Subtitle**: Professional, modern, clean process diagram
- Starts with User Query (user asks a question)
- **Next step**: Understand Intent (AI interprets the request)
- Process splits into two parts:
  - Retrieve Knowledge (RAG - Policies + QA) (fetches relevant insurance info)
  - Analyze Data (Structured Insights) (processes structured data)
- Both paths merge into Generate Explanation (AI creates a response)
- **Then:** Create Infographic (visual representation of insights)
- **Final step**: Final Output (delivered to the user)



2.4 Components Design

- The diagram shows a multimodal AI system that processes a user query to generate both explanation and infographic.
- The UI sends the query to the Orchestrator, which controls the overall workflow.
- The Agent Manager coordinates all agents like Query, Retrieval, Data, Explanation, and Infographic agents.
- The RetrievalAgent and DataAgent fetch information from PolicyDocs, VectorDB, and Dataset.
- The ExplanationAgent uses LLM to generate a clear textual explanation.
- The InfographicAgent uses ImageGenerator to create a visual infographic.
- The system combines RAG, data analysis, and multimodal generation to produce final outputs.



2.5 Key Design Considerations

- **Scalability:**
  - Modular architecture allows horizontal scaling of AI agents and backend API for multiple concurrent users.

- Vector DB and LLM API calls can be distributed to handle high traffic.

- **Performance:**
  - Efficient use of caching for repeated queries to reduce latency.
  - Asynchronous API calls to LLM and Image Generation services to improve responsiveness.
- **Reliability:**
  - Retry mechanisms for external API calls.
  - Graceful handling of partial failures (e.g., text generated but image API fails).
- **Security:**
  - Secure storage of API keys.
  - Input validation to prevent malicious prompts.
  - HTTPS for all client-server communications.
- **Cost Optimization:**
  - Use of token-efficient queries to LLM API.
  - Optional image resolution or batching to reduce API costs.

2.6 API Catalogue

1. Internal API: Generate Multimodal Output

Endpoint: POST /generate

Description: Accepts a user query and returns a comprehensive multimodal response, including both textual explanation and a visual infographic.

➤ **Request Example:**

JSON

```
{
"query": "Explain flood risk for coastal properties"
}
```

➤ **Response Example:**

JSON

{

"narrative": "Flood risk includes storm surges, heavy rainfall, and rising sea levels. Proper insurance coverage is recommended to mitigate financial losses.", "image_url": "https://your-storage.com/generated-infographic.png"

}

2. Internal API: Health Check

Endpoint: GET /health

Description: Returns the status of the backend service to ensure the system is operational.

- **Response Example:**

JSON

{

"status": "OK"

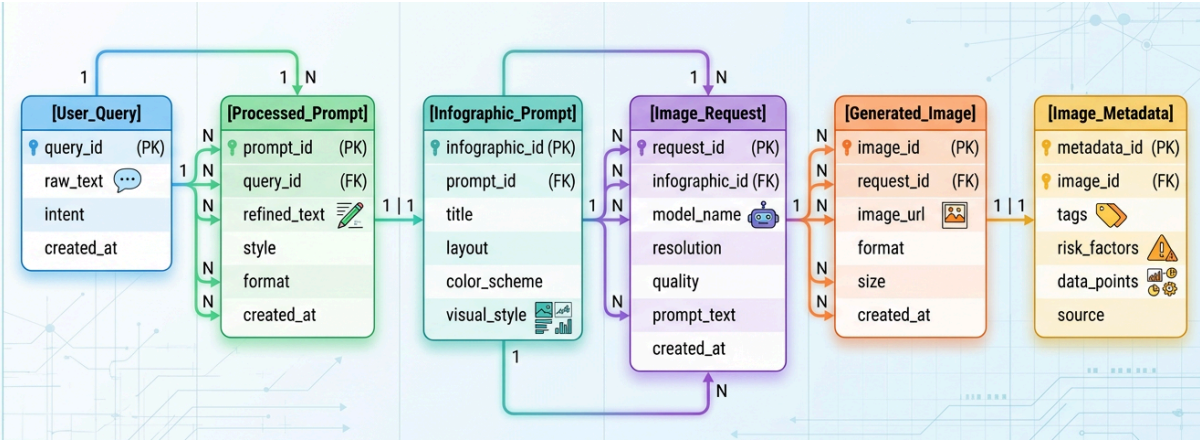
}

3. API USED

API Name	Purpose	Description
LLM API	Text Generation	Used by the Explanation Generation Agent to produce detailed textual narratives.
Image Generation API	Infographic/Image Generation	Used by the Image Generation Model to create visual representations based on text prompts.
Vector Database API	Context Retrieval	Used by the RAG Retrieval Agent to fetch relevant contextual data using embeddings.

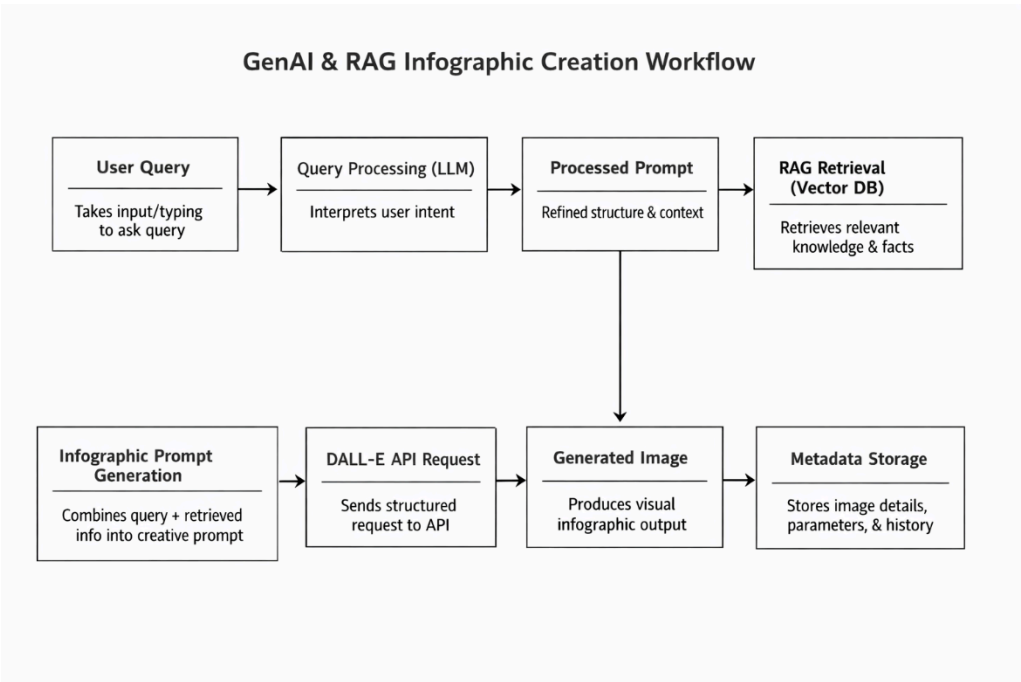
3. Data Design

3.1 Data Model



The data model supports the full pipeline from user query to generated infographic image. Each stage of processing is stored as a separate entity to ensure traceability, scalability, and auditability. Relationships ensure that one query can generate multiple prompts and multiple image outputs.

3.2 Data Access Mechanism



The system follows a layered data access approach:

- Structured data is accessed via SQL/Pandas for analytics.
- Unstructured data is retrieved using vector similarity search (RAG).
- DALL·E API is used for image generation.
- Each component communicates via APIs ensuring modularity.

3.3 Data Retention Policies

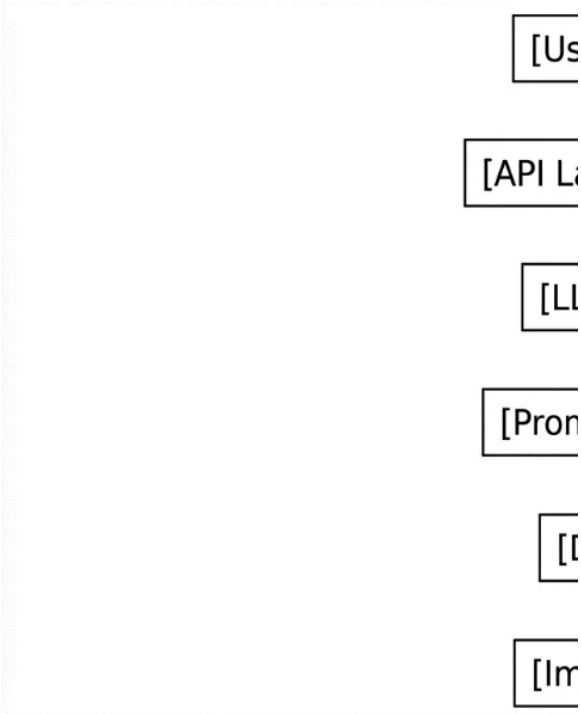
- User Queries: Stored temporarily for session continuity; optionally logged for analytics.
- Prompts and Infographic Data: Retained for reproducibility and debugging.
- Generated Images: Stored for short-term use (24–72 hours) unless explicitly saved.
- Metadata: Retained longer for audit and explainability.
- Compliance:
    - Sensitive data should be anonymized.
    - Follow data protection standards (e.g., GDPR principles if applicable).

3.4 Data Migration

Data migration ensures system adaptability to evolving models and datasets.

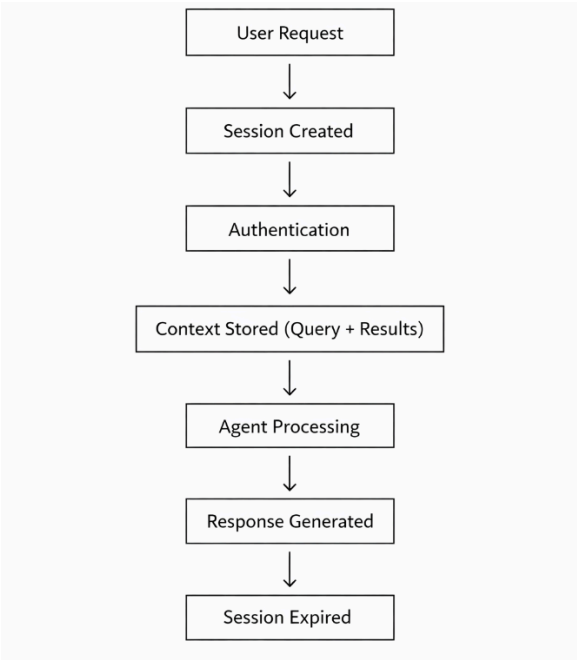
- **Schema Migration:**
    - Managed using version control tools (e.g., Alembic).
- **Embedding Migration:**
    - Recompute embeddings when models are updated.
- **Prompt Evolution:**
    - Maintain backward compatibility for older prompts.
- **Zero-Downtime Strategy:**
    - Use parallel systems during migration to avoid service disruption.

4. Interfaces



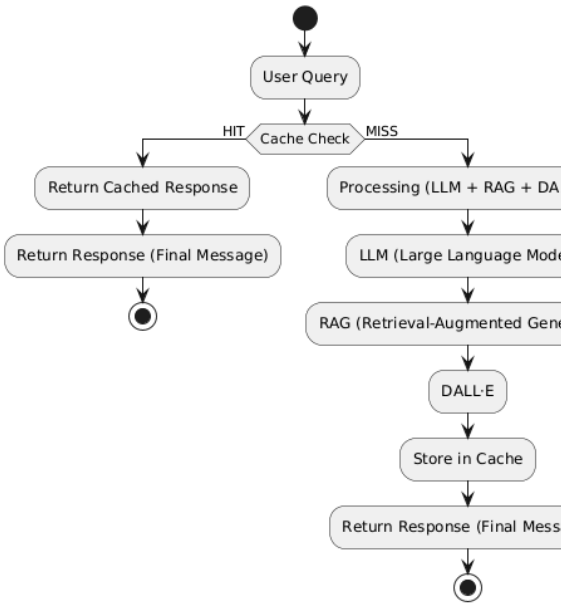
- The system provides a web-based interface for user interaction.
- Backend APIs handle communication with:
  - LLM services
  - Vector databases
  - Image generation APIs
- Output includes both text explanations and infographic images.

5. State and Session Management



- **Sessions maintain:**
  - Query history
  - Retrieved documents
  - Generated prompts
- **Stored using:**
  - Redis (recommended) or in-memory storage
- **Enables context-aware responses**

6. Caching



Caching improves performance and reduces API cost:

- **Query Cache**: Stores repeated queries
- **Embedding Cache**: Avoids recomputation
- **LLM Response Cache**: Stores generated explanations
- **Tools**: Redis / in-memory cache

7. Non-Functional Requirements

7.1 Security Aspects

 Data Encryption:

- At rest (AES-256)
- In transit (HTTPS)

 Authentication & Authorization:

- JWT / OAuth-based access

 Data Privacy:

- Mask sensitive insurance data

 Prompt Security:

- Input validation to prevent prompt injection

 API Security:

- Rate limiting and request validation

7.2 Performance Aspects

 Latency Targets:

- Text response: < 5 seconds
- Image generation: < 10 seconds

 Scalability:

- Horizontal scaling of API and vector DB

 Optimization Techniques:

- Parallel agent execution
- Batch processing
- Efficient caching

 Monitoring:

- Track response time, errors, and throughput

8. References

1. OpenAI. GPT-4 Technical Report. 2023. https://arxiv.org/abs/2303.08774

– Details the architecture and capabilities of large language models.

2. Ramesh, A., et al. Zero-Shot Text-to-Image Generation with DALL·E. NeurIPS, 2021. https://arxiv.org/abs/2102.12092

– Discusses text-to-image generative modeling techniques.

3. Brown, T., et al. Language Models are Few-Shot Learners. NeurIPS, 2020. https://arxiv.org/abs/2005.14165

– Foundational paper on LLM capabilities.

4. Radford, A., et al. Learning Transferable Visual Models From Natural Language Supervision (CLIP). ICML, 2021. https://arxiv.org/abs/2103.00020

– Explains alignment of text and image embeddings.

5. Karras, T., et al. A Style-Based Generator Architecture for Generative Adversarial Networks. CVPR, 2019. https://arxiv.org/abs/1812.04948

– Key reference for high-quality image generation techniques.